National University of Computer and Emerging Sciences

DL-2001: Introduction to Data Science
Lab Manual 09

Department of Data Science
FAST-NU, Lahore, Pakistan

Objectives

- To implement Evaluation metrics for regression - Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-squared (R^2), and Adjusted R-squared from scratch.
- To validate correctness by comparing results with Scikit-learn's metric functions.

- To manually compute and interpret evaluation metrics for classification:
  - Confusion Matrix
  - Precision
  - Recall
  - F1-score
- To verify manually computed results using Scikit-learn's built-in methods.

Evaluation Metrics for Linear Regression

Linear Regression is a supervised learning algorithm used to model the relationship between a dependent variable y and one or more independent variables X.

Equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

where:

- $\beta_0, \beta_1, \ldots, \beta_n$ are model coefficients,

After fitting a linear regression model, we must evaluate its accuracy using quantitative measures.

4.1 Mean Absolute Error (MAE)

Measures the average magnitude of errors without considering direction.

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

- **Interpretation:** Lower MAE → better fit.
- **Units:** Same as the dependent variable y.

4.2 Mean Squared Error (MSE)

Calculates the average squared difference between predicted and actual values.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Penalizes large errors more heavily.
- **Lower MSE → better performance.**

4.3 Root Mean Squared Error (RMSE)

Square root of MSE; brings metric back to the same units as y.

$$RMSE = \sqrt{MSE}$$

- **Interpretation:** More sensitive to outliers than MAE.
- **Lower RMSE → better accuracy.**

4.4 Coefficient of Determination (R^2)

Indicates how well the independent variables explain the variability of the dependent variable.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

- **Range:** $0 \le R^2 \le 1$
- **Interpretation:** Closer to 1 means better fit.

TASK1:

Data Preparation

1. Generate a **synthetic regression dataset** using NumPy.

```
import numpy as np

# Set random seed for reproducibility

np.random.seed(42)
```

```
# Generate synthetic data

X = 2 * np.random.rand(100, 1)        # 100 samples, 1 feature

y = 4 + 3 * X.flatten() + np.random.randn(100)  # Linear relation with noise
```

2. Display your generated data

```
print("Feature values (X):")

print(X[:10])   # show first 10 samples



print("\nTarget values (y):")

print(y[:10])   # show first 10 target values
```

3. Split the dataset into **training** and **testing** sets.

Model Training

1. Train a **Linear Regression** model using `scikit-learn`.
2. Generate predictions on the test set.

Implement Metrics from Scratch

Write Python functions (using only **NumPy**) to calculate the following metrics:

1. Mean Absolute Error (MAE)
2. Mean Squared Error (MSE)
3. Root Mean Squared Error (RMSE)
4. R-squared (R^2)

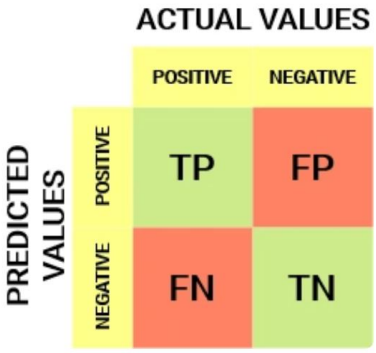
Verify Using Built-in Methods

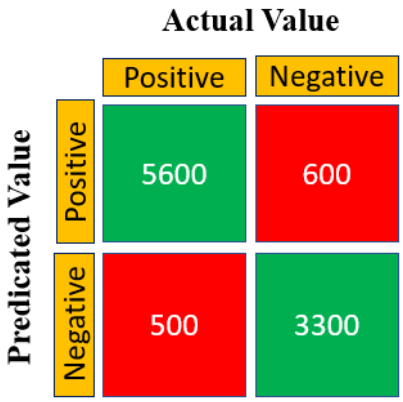
Use Scikit-learn's built-in functions to compute the same metrics.

Compare Results

Display both **manual** and **built-in** results side-by-side to verify correctness.

Evaluation Metrics for Classification





Classification Evaluation Metrics

1. **Confusion Matrix**

   A 2x2 table summarizing the performance of a classification model by comparing predicted vs actual classes.

   - **True Positive (TP):** Correctly predicted positive class
   - **True Negative (TN):** Correctly predicted negative class
   - **False Positive (FP):** Incorrectly predicted positive (Type I error)
   - **False Negative (FN):** Incorrectly predicted negative (Type II error)

2. **Precision**

   Measures how many of the predicted positive instances are actually positive.

   $$\text{Precision} = \frac{TP}{TP + FP}$$

   High precision means fewer false positives.

3. **Recall (Sensitivity or True Positive Rate)**

   Measures how many of the actual positive instances were correctly predicted.

   $$\text{Recall} = \frac{TP}{TP + FN}$$

   High recall means fewer false negatives.

4. **F1-score**

   The harmonic mean of Precision and Recall, providing a balance between the two.

   $$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

   Useful when you want to balance false positives and false negatives.

5. **ROC Curve (Receiver Operating Characteristic Curve)**

   A plot of **True Positive Rate (Recall)** vs **False Positive Rate (FPR)** at various classification thresholds.

   It illustrates the trade-off between sensitivity and specificity.

6. **AUC (Area Under the Curve)**

   A scalar value summarizing the ROC curve's performance.

   - Ranges from 0 to 1
   - AUC = 1 means perfect classification
   - AUC = 0.5 means no better than random guessing

TASK2:

1. Load the **Iris dataset** using Scikit-learn.
2. Convert it into a **binary classification** problem:
   Predict whether a flower is *Iris-Virginica (1)* or *not Virginica (0)*.

```
# Load dataset

iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)

y = (iris.target == 2).astype(int)  # Virginica = 1, else 0
```

1. Split the data into **training (80%)** and **testing (20%)** sets.
2. Use KNN to make predictions. K=3
3. Implement confusion matrix from scratch.

Term	Meaning
TP	Correctly predicted positive class (Virginica)
TN	Correctly predicted negative class (Not Virginica)
FP	Incorrectly predicted positive
FN	Incorrectly predicted negative

4. Using the matrix, find Accuracy, Precision, Recall,F1 from scratch.
5. Verify values for these metrics Using Built-in Methods.